

Is it possible to build?

Absolutely, YES. In fact, this is the exact architecture that top-tier AI companies are using right now (often called "Structured Output" or "UI as Code").

For a 10-day hackathon, you do not need the AI to write actual React/Flutter code on the fly (which would be impossible offline). Instead, you pre-build the physical widgets (the `Macro_Grid`, the `Vision_Scanner`, the `Checklist`) into your app's frontend.

When the local Gemma 4 LiteRT model processes the user's prompt, you enforce a **JSON output schema**. The AI simply outputs a tiny JSON string like `{"render": "macro_grid", "items": ["Nasi Lemak", "Teh O"]}`. Your app reads that string, hides the chat box, and renders the pre-built grid component. It is incredibly elegant, completely offline, and 100% feasible for an MVP.

The Single-App Architecture (Bina.ai)

Putting all three workspaces into a single app is a brilliant product decision. It mirrors apps like TikTok or Instagram, where users can seamlessly switch between consuming content and creating it.

Here is the complete, detailed UI/UX breakdown for the unified **Bina.ai App**, separated by a simple Role-Toggle switch.

The Onboarding & Role Toggle

When a user opens Bina.ai for the first time, they are met with a clean, localized welcome screen.

- **The Switch:** A simple profile toggle in the settings menu: **[Switch to Architect Mode]** or **[Switch to Builder Mode]**.
- **The Navigation Bar:** The bottom navigation bar changes depending on the active role.
 - *Builder Nav:* **[Hub]** | **[My Pocket]** | **[Offline Sync]**
 - *Architect Nav:* **[Hub]** | **[Studio]** | **[Analytics]**

Workspace 1: The Recipe Hub (Shared by Both)

This is the default landing page. It acts as the decentralized "App Store" for offline intelligence.

UI & Features:

- **The Hero Carousel:** Highlights critical, verified Recipes (e.g., *"NADMA Official: Offline Flood Triage Agent"*).
- **Smart Search & Filter Tags:** Large, tap-friendly tags to filter by Domain (Agriculture, Education, Finance) or Supported Dialects (Kelantanese, Iban, Taglish).

- **The Recipe Detail Card:** When a user taps a Recipe, it slides up a card displaying:
 - **Credibility Badge:** Architect's name with a Blue Tick (e.g., *Verified by Ministry of Health*).
 - **Data Footprint:** Shows the exact offline size (e.g., *Recipe Size: 1.2KB*).
 - **Generative UI Preview:** A looping GIF or static mockup showing what widgets this Recipe usually generates (e.g., showing the POS buttons).
 - **The Action Button:** A massive **[Install to Pocket]** button.

Workspace 2: The Studio (Architect Mode)

Since typing raw JSON or Markdown on a mobile phone is frustrating, the mobile Studio acts as a guided "Logic Forge." It allows experts to build complex `skills.md` files using simple mobile forms.

UI & Features:

- **The "Draft Recipe" Interface:**
 - **Step 1: Identity (The Persona):** A text box where the Architect writes the core rules. (e.g., *"You are an agricultural advisor. Focus only on palm oil."*)
 - **Step 2: Knowledge Injection:** A button to **[Upload PDF/Text]**. The Architect uploads a MARDI fertilizer document. The app parses it into the Recipe's offline knowledge base.
 - **Step 3: Widget Whitelisting (The Arsenal):** A checklist of capabilities the Architect *allows* the AI to use. They toggle on: ☒ Allow Camera Vision, ☒ Allow Macro Grid, ☐ Allow SMS Dispatch.
- **The "Simulator" Tab:**
 - Before publishing, the Architect swipes right to a "Test Canvas." They can type a mock prompt as if they were a Builder to see if the Gemma model successfully triggers the Generative UI offline.
- **The Publish Button:** Once tested, tapping **[Publish to Hub]** compiles the inputs into the 2KB JSON/MD Recipe and pushes it to the public directory.

Workspace 3: The Pocket (Builder Mode)

This is the execution zone. It is a hybrid engine: it uses AI to generate the first draft of the UI instantly, but gives the Builder the ultimate authority to manually edit, add, or delete widgets.

UI Layout & Features:

1. The Agent Carousel (Top Bar)

- A horizontal scrolling list of installed Pocket Agents (e.g., the *Buku Kira-Kira*, the *Bidan Pintar*). Tapping an icon instantly switches the UI context to that specific offline Recipe.

2. The Hybrid Canvas (Middle Screen)

- **The AI's First Draft (Generative UI):** The Builder inputs their request ("*Saya nak jual menu pagi.*"). The LiteRT model dynamically generates the POS **Macro_Grid** and renders it on the screen.
- **The Builder's Override (Manual Customization):** The Builder looks at the AI's layout and decides they want to tweak it. They tap a prominent [ **Ubah Suai / Customize**] toggle at the top right of the canvas.
- **The "Jiggle" Edit Mode:** The screen enters a customization state (similar to rearranging apps on an iOS/Android home screen).
 - **Tap to Edit:** The Builder taps an AI-generated button (e.g., *Nasi Lemak*). A simple bottom-sheet slides up allowing them to manually rename the label, change the color to red, or adjust the underlying price value.
 - **Drag to Delete:** They can drag an irrelevant widget into a trash can icon to permanently remove it from their layout.

3. The Local Component Library (Adding New Widgets)

- While in "Customize" mode, a floating [**+ Tambah Alat / Add Widget**] button appears.
- If the AI missed something crucial, the Builder taps this button and opens a simplified, picture-based menu of offline widgets they can manually drop into their Pocket Agent:
 - *Add a Button (Macro_Grid item)*
 - *Add a Checklist (ActionList)*
 - *Add a Camera Scanner (Vision_EdgeScanner)*
- **Use Case Example:** The AI generated a great POS system for the *Makcik*, but she also wants a quick way to track her own daily mood. She taps [**+**], selects the **Analog_Context_Slider**, and labels it "*Berapa penat hari ini?*" (How tired today?). She has successfully customized her financial app to include personal wellness tracking, completely bypassing the AI.

4. The "Pin UI State" Protocol

- Once the Builder is happy with their manual adjustments, they tap [**Simpan / Save**].
- Bina.ai takes this heavily customized UI layout and overwrites the AI's default generation rules for that specific device. The next time the Builder opens this Pocket Agent, it instantly loads their perfect, hand-crafted layout without needing to reprompt the AI.

5. The Dynamic Input Dock (Bottom Bar)

- Anchored to the bottom of the screen.
- Defaults to a massive **[Voice Record]** button for low-literacy users.
- Includes a **[Keyboard]** toggle for those who prefer to type, and a **[Camera]** trigger icon to feed visual data directly into the active UI widgets.

How to Build This for the MVP (The Developer Plan)

To actually pull this off in the hackathon, here is your technical game plan:

1. **Frontend:** Use Flutter or React Native. Build ~5 hardcoded UI components (ChatBubble, MacroGrid, Checklist, DualPane, Slider).
2. **The Parser:** Write a simple middleware function. When the LiteRT model finishes generating, your function checks if the output is standard text or a JSON block.
3. **The Trigger:** If the model outputs `{"type": "Checklist", "data": ["Step 1", "Step 2"]}`, your frontend hides the ChatBubble component and displays the Checklist component, feeding it that data array.

You have just designed an edge-native, Generative UI operating system inside a single mobile app. Does this single-app flow give you a clear enough roadmap to start drawing up the Figma wireframes or drafting the codebase architecture?